#HACKTHEKITTY 2026 · PROJECT REPORT

MeoW — Cat Safety Network

A safety net that makes sure your cat is always cared for — even when you're not there.

Project: MeoW **Reference ID:** 6AOYB23P **Date:** 7 July 2026

1. Executive Summary

MeoW is a cat-safety network that makes sure a cat is never left without a plan. Every cat carries a QR tag and a prioritised chain of trusted guardians, so the right person is contacted — instantly and automatically — whether a stranger finds a lost cat or the owner becomes unreachable. Recovery and care escalations run on durable Temporal workflows, so the safety net keeps working reliably in the background even across restarts.

2. Project Overview

2a. Why we're building this

When a cat goes missing or its owner suddenly can't provide care (travel, hospital, an emergency), there is usually **no automatic fallback**. A microchip only helps if the cat reaches a vet; a phone number on a collar only helps if the owner happens to answer. Nothing escalates on its own, and nobody with the cat's care details is looped in. MeoW closes that gap with two automatic escalation paths and a shared source of truth for each cat's care instructions.

2b. How it relates to the theme

The theme is **#hackthekitty** — cats. MeoW is built end to end around cat welfare: cat profiles, cat recovery via QR tags, and a guardian network whose entire purpose is keeping cats cared for. The theme is the core of the product, not a surface-level skin.

2c. Target Audience

- **Cat owners** who want peace of mind that their cat is covered if it's lost or if they can't be reached.
- **Guardians** (friends, family, neighbours, sitters) who step in during an emergency and need the cat's care details fast.
- **Finders** — any member of the public who finds a cat and scans its tag; no account needed.

3. Key Features

- ✓ **QR-tag lost-cat recovery** — a finder scans a cat's tag, sees the owner's contact info, and submits their own details; the owner is alerted by email.
- ✓ **Auto-escalating guardian chain** — if the owner doesn't respond in 10 minutes, guardians are notified in priority order automatically.

- ✓ **Owner "unavailable" mode** — the owner triggers the chain manually; each guardian gets a 30-minute window before the next is contacted.
- ✓ **Guardian magic-link access** — whoever accepts gets a secure, time-limited link to the cat's full care instructions.
- ✓ **AI care assistant** — a Groq-powered assistant answers guardians' care questions, scoped strictly to that cat's data, with prompt-injection defences.
- ✓ **Cat & guardian profiles** — rich profiles (diet, meds, vet, personality) with image uploads to Cloudinary.
- ✓ **Durable workflows** — all escalations run on Temporal so timers and retries survive restarts.
- ✓ **QR code simulator** — a built-in demo tool to walk the scan flow without a physical tag.

4. Technology Stack

LAYER	TECHNOLOGY
Frontend / templating	express-handlebars, vanilla JS, CSS
Backend	Node.js 20 (ES modules), Express 5
Database	MongoDB (native driver)
Authentication	Auth0 (password-realm grant) + MongoDB-backed session layer
Durable workflows	Temporal (Temporal Cloud)
Email	Nodemailer (Gmail SMTP)
Image hosting	Cloudinary (uploads via multer)
AI assistant	Groq (`llama-3.1-8b-instant`)
QR generation	`qrcode`
Security	helmet (CSP + nonces), `csrf-csrf` , `express-rate-limit`

5. Technical Architecture

The application follows a strict **three-layer separation**, with a separate **Temporal worker** process running the durable workflows. All application code lives under `src/` (`src/index.js` , `src/business.js` , `src/temporal/` , `src/views/` , `src/public/`).

```
Browser / Finder
      |
 index.js          routes, CSP, CSRF, rate limits
      |
 presentation.js   request shaping / pass-through
      |
 business.js       domain logic, ownership checks, workflow triggers
      |
 persistance.js -> MongoDB   (only layer touching the DB)
```

```
business.js -> temporal/client -> Temporal Cloud
                                        |
                                temporal/worker.js  (workflows + activities)
                                        |
                        activities -> mailer.js (Gmail SMTP)

External services: Auth0 (auth) . Cloudinary (images) . Groq (AI chat)
```

Finder scenario. A finder opens `/scan?qr=<id>` → `index.js` validates the QR format → `business.getCatInfoForScan` looks up cat and owner → the finder submits details → `business.handleScan` creates an emergency event, emails the owner, and starts a `foundCatWorkflow`. The workflow waits 10 minutes for the owner to acknowledge (a Temporal **signal**); if it never arrives, it notifies each guardian in priority order.

Owner-unavailable scenario. The owner posts to `/owner/unavailable` → `business.setOwnerUnavailable` starts an `ownerUnavailableWorkflow` that emails each guardian a magic link in priority order, waiting 30 minutes per guardian, and stops as soon as one accepts.

Key technical decisions

- **Temporal for durable execution** — long timers (10–30 min) must survive restarts and deliver exactly once; a naive `setTimeout` would be lost on restart.
- **Auth delegated to Auth0** — the app never stores or compares passwords; it keeps only a thin session cookie.
- **Three-layer boundary** — only `persistance.js` touches MongoDB, centralising ownership checks in `business.js`.
- **Cloudinary for images** — offloads binary storage and delivery from the app and database.

6. Testing Matrix

Manual test cases run against a local build (Node 20, connected to MongoDB and Temporal Cloud).

FEATURE / FLOW	STEPS	EXPECTED	ACTUAL	RESULT
Server boot	`npm start`	Connects to MongoDB, listens on :3000	Connected + listening	PASS
Landing page	Visit `/`	Both scenarios render	Renders (200)	PASS
Register	Submit register form	Account created via Auth0	Account created	PASS
Login / session	Valid credentials	Session cookie set, redirect	Logged in	PASS
Add cat	Add a cat with photo	Saved, photo on Cloudinary, QR ID	Cat created	PASS
Ownership guard (IDOR)	Edit another user's cat ID	"Cat not found" — no access	Access denied	PASS
QR scan lookup	Open `/scan?qr=<id>`	Cat + owner + finder form	Shown (200)	PASS
Finder submit	Submit finder details	Event created, owner emailed	Alert sent	PASS
Owner acknowledgment	Click ack link	Escalation stopped	Stopped	PASS
Guardian escalation	Leave scan unacknowledged	Guardian emailed after wait	Notified	PASS
Owner unavailable	Mark unavailable	Guardian chain starts	Chain started	PASS
AI care assistant	Ask a care question	Scoped answer; injection refused	Answered / refused	PASS
Rate limit (finder)	Submit `/scan` repeatedly	Throttled after limit	Throttled	PASS
CSRF-safe logout	Log out	POST with token; cross-site cannot	Logged out	PASS
Account deletion	Delete account	Auth0 identity removed; no re-login	Removed	PASS
QR simulator	Paste QR ID, Simulate scan	Redirects to finder page	Redirected	PASS

7. Security Scan (Aikido)

The project is connected to **Aikido** and scanned on every push. All findings — NoSQL injection (Critical), Server-Side Template Injection (High), unescaped-template XSS (High), a brace-expansion dependency DoS (Medium), and an open redirect (Low) — are **fixed and confirmed Solved on re-test**. Full detail and the controls in place are in the accompanying **Security Report**; the threat model is in the **Threat Model** document. Scan screenshots are attached with the submission.

8. Future Improvements

- **SMS / push notifications** in addition to email, for faster response.
- **Printable / orderable physical tags** (and NFC) generated from each cat's QR.
- **Automated test suite** (unit + integration) to replace the manual matrix.
- **Guardian mobile view / app** with one-tap accept.
- **Session revocation on password change** and email-bound guardian tokens (from the threat model roadmap).

9. Tools Used

- **Claude Code** — AI pair-programming assistant used throughout development.
- **VS Code** — editor / IDE.
- **Temporal Cloud, Auth0, Cloudinary, Groq** — managed platforms integrated into the app.
- **Aikido** — security scanning (SAST + AI code audit).
- **Git / GitHub** — version control and hosting.

10. Learnings & Takeaways

- **Durable execution changes how you model time.** Moving escalation timers into Temporal workflows made the logic both simpler and far more reliable — restarts and retries stopped being a concern.
- **Delegating auth pays off.** Using Auth0 removed a whole class of password and credential-handling risk.
- **Security is a design activity, not a final step.** Ownership checks, CSP nonces, CSRF, and rate limiting were most effective built into each layer as we went; the Aikido scan then confirmed the remaining edge cases were closed.

11. Acknowledgments

- **Temporal** for durable workflow execution.
- **Auth0** for authentication; **Cloudinary** for image hosting; **Groq** for fast LLM inference.
- The open-source maintainers of Express, the MongoDB Node driver, express-handlebars, helmet, csrf-csrf, nodemailer, multer, and qrcode.

Submission Checklist

- ✓ README (prerequisites, run instructions, configuration)
- ✓ Project report (this document, in `documentation/`)
- ✓ Security report + threat model (in `documentation/`)
- ✓ Aikido security scan — all issues Solved (screenshots attached)
- ✓ Source code in `src/`
- ✓ No committed `node_modules/`, build output, or unrelated files
- ✓ Video demo (HD, 720p or higher)